

**Documento:Uso de Índices**

Capturas De Pantalla: De Comandos Ejecutados

Nombre De Autores

Dina Arely Tino De Martínez

Marlon Gerardo Flores Portillo

**Universidad Doctor Andres Bello Reguinal Sonsonate.**

Nombre: Del Módulo.

**Módulo III : Bases De Datos No Relacionales**

**Catedrático**

**Ingeniero:Jose Rosalio Serrano**

**Sonsonate, 23 /05/2026**

### Indicaciones:

- Crea una BD con nombre unab
- Crea una colección llamada: empleados
- realiza 20 registros con el formato siguiente:

```
{ nombre: "Carolina Suárez",
```

```
edad: 29,
```

```
ciudad: "Bogotá",
```

```
correo: "carolina@unab.com",
```

```
cargo: "Desarrolladora Backend",
```

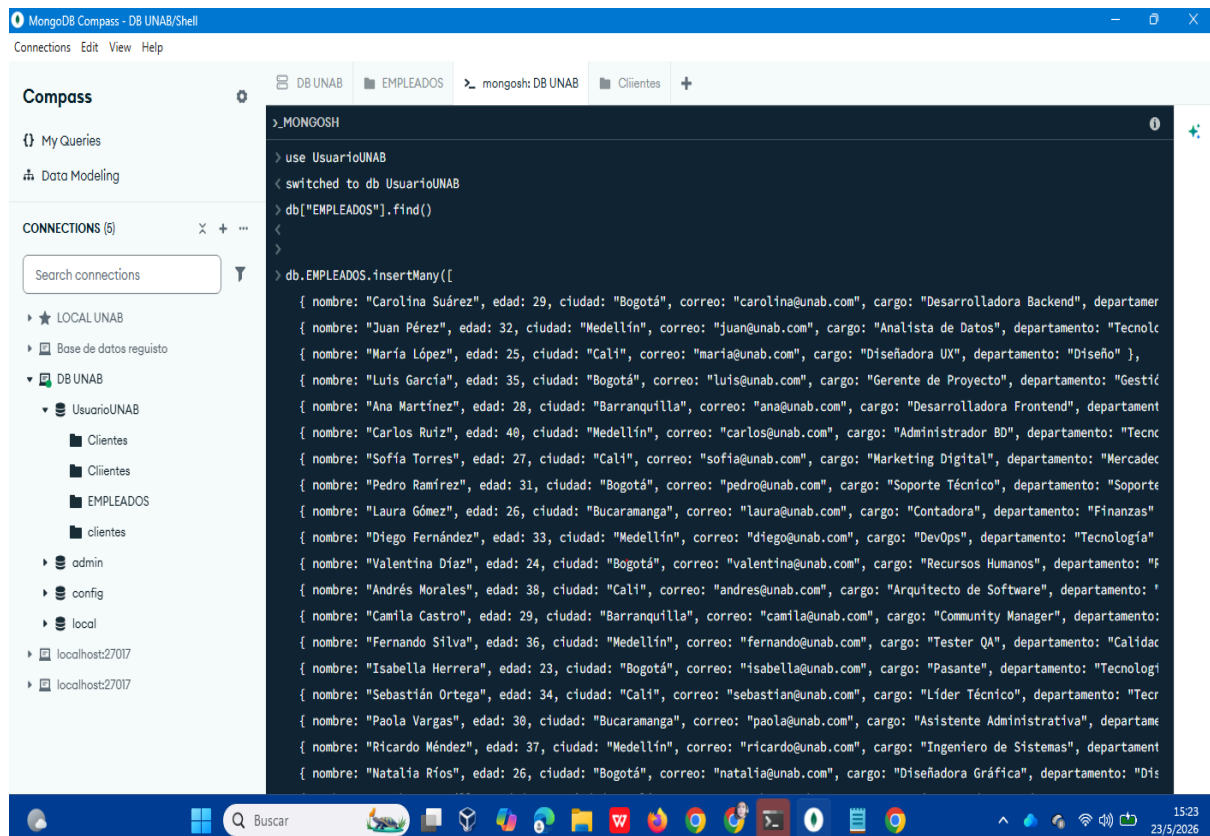
```
departamento: "Tecnología"
```

```
}
```

### Evidencia de códigos

1)

- Crea una BD con nombre unab
- Crea una colección llamada: empleados



## Especificación de las capturas de pantalla

use Usuario UNAB

switched to db UsuarioUNAB

db["EMPLEADOS"].find()

db.EMPLEADOS.insertMany([

```

    { nombre: "Carolina Suárez", edad: 29, ciudad: "Bogotá", correo:
"carolina@unab.com", cargo: "Desarrolladora Backend", departamento: "Tecnología"
},

```

```

    { nombre: "Juan Pérez", edad: 32, ciudad: "Medellín", correo: "juan@unab.com",
cargo: "Analista de Datos", departamento: "Tecnología" },

```

```

    { nombre: "María López", edad: 25, ciudad: "Cali", correo: "maria@unab.com",
cargo: "Diseñadora UX", departamento: "Diseño" },

```

{ nombre: "Luis García", edad: 35, ciudad: "Bogotá", correo: "luis@unab.com",  
cargo: "Gerente de Proyecto", departamento: "Gestión" },

{ nombre: "Ana Martínez", edad: 28, ciudad: "Barranquilla", correo:  
"ana@unab.com", cargo: "Desarrolladora Frontend", departamento: "Tecnología" },

{ nombre: "Carlos Ruiz", edad: 40, ciudad: "Medellín", correo: "carlos@unab.com",  
cargo: "Administrador BD", departamento: "Tecnología" },

{ nombre: "Sofía Torres", edad: 27, ciudad: "Cali", correo: "sofia@unab.com",  
cargo: "Marketing Digital", departamento: "Mercadeo" },

{ nombre: "Pedro Ramírez", edad: 31, ciudad: "Bogotá", correo:  
"pedro@unab.com", cargo: "Soporte Técnico", departamento: "Soporte" },

{ nombre: "Laura Gómez", edad: 26, ciudad: "Bucaramanga", correo:  
"laura@unab.com", cargo: "Contadora", departamento: "Finanzas" },

{ nombre: "Diego Fernández", edad: 33, ciudad: "Medellín", correo:  
"diego@unab.com", cargo: "DevOps", departamento: "Tecnología" },

{ nombre: "Valentina Díaz", edad: 24, ciudad: "Bogotá", correo:  
"valentina@unab.com", cargo: "Recursos Humanos", departamento: "RRHH" },

{ nombre: "Andrés Morales", edad: 38, ciudad: "Cali", correo: "andres@unab.com",  
cargo: "Arquitecto de Software", departamento: "Tecnología" },

{ nombre: "Camila Castro", edad: 29, ciudad: "Barranquilla", correo:  
"camila@unab.com", cargo: "Community Manager", departamento: "Mercadeo" },

{ nombre: "Fernando Silva", edad: 36, ciudad: "Medellín", correo:  
"fernando@unab.com", cargo: "Tester QA", departamento: "Calidad" },

{ nombre: "Isabella Herrera", edad: 23, ciudad: "Bogotá", correo:  
"isabella@unab.com", cargo: "Pasante", departamento: "Tecnología" },

```
{ nombre: "Sebastián Ortega", edad: 34, ciudad: "Cali", correo: "sebastian@unab.com", cargo: "Líder Técnico", departamento: "Tecnología" },
```

```
{ nombre: "Paola Vargas", edad: 30, ciudad: "Bucaramanga", correo: "paola@unab.com", cargo: "Asistente Administrativa", departamento: "Administración" },
```

```
{ nombre: "Ricardo Méndez", edad: 37, ciudad: "Medellín", correo: "ricardo@unab.com", cargo: "Ingeniero de Sistemas", departamento: "Tecnología" },
```

```
{ nombre: "Natalia Ríos", edad: 26, ciudad: "Bogotá", correo: "natalia@unab.com", cargo: "Diseñadora Gráfica", departamento: "Diseño" },
```

```
{ nombre: "Esteban Castillo", edad: 39, ciudad: "Cali", correo: "esteban@unab.com", cargo: "Director de TI", departamento: "Tecnología" }
```

```
])
```

```
{
```

```
  acknowledged: true,
```

```
  insertedIds: {
```

```
    '0': ObjectId('6a1213d2df4be4efc675a113'),
```

```
    '1': ObjectId('6a1213d2df4be4efc675a114'),
```

```
    '2': ObjectId('6a1213d2df4be4efc675a115'),
```

```
    '3': ObjectId('6a1213d2df4be4efc675a116'),
```

```
    '4': ObjectId('6a1213d2df4be4efc675a117'),
```

```
    '5': ObjectId('6a1213d2df4be4efc675a118'),
```

```
    '6': ObjectId('6a1213d2df4be4efc675a119'),
```

```
    '7': ObjectId('6a1213d2df4be4efc675a11a'),
```

```
    '8': ObjectId('6a1213d2df4be4efc675a11b'),
```

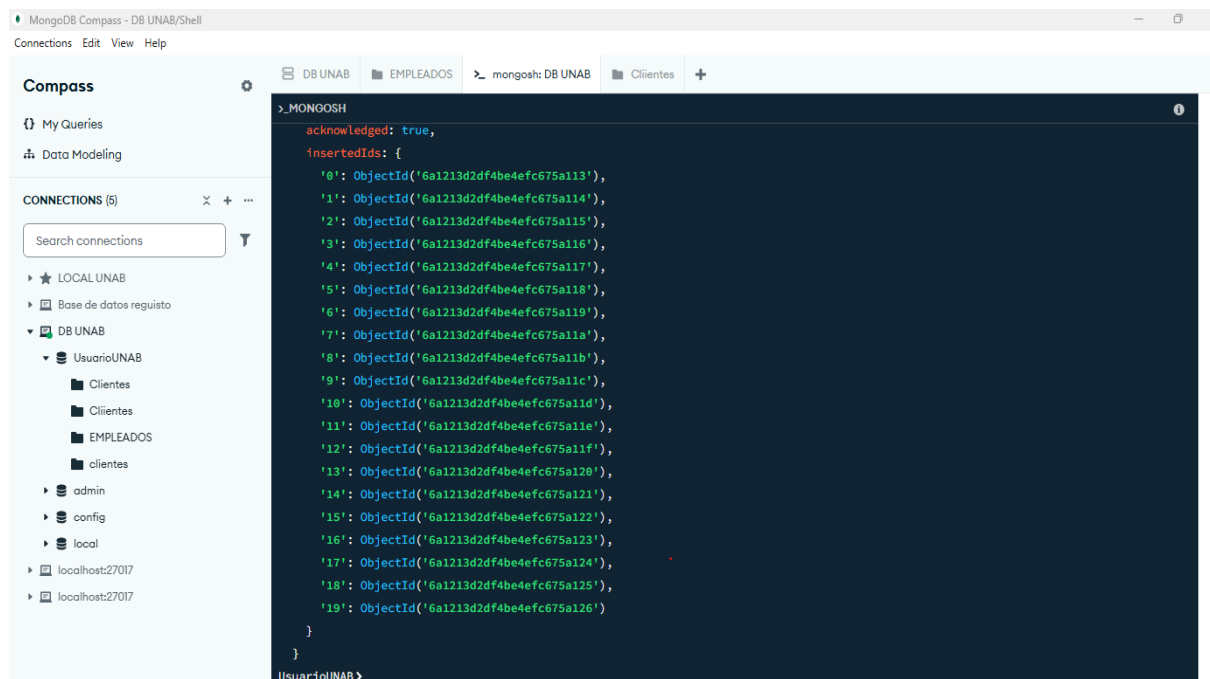
```

'9': ObjectId('6a1213d2df4be4efc675a11c'),
'10': ObjectId('6a1213d2df4be4efc675a11d'),
'11': ObjectId('6a1213d2df4be4efc675a11e'),
'12': ObjectId('6a1213d2df4be4efc675a11f'),
'13': ObjectId('6a1213d2df4be4efc675a120'),
'14': ObjectId('6a1213d2df4be4efc675a121'),
'15': ObjectId('6a1213d2df4be4efc675a122'),
'16': ObjectId('6a1213d2df4be4efc675a123'),
'17': ObjectId('6a1213d2df4be4efc675a124'),
'18': ObjectId('6a1213d2df4be4efc675a125'),
'19': ObjectId('6a1213d2df4be4efc675a126')
}
}

```

**Realiza 20 registros con el formato siguiente:**

**use Usuario UNAB**



## 2) INDICACIONES

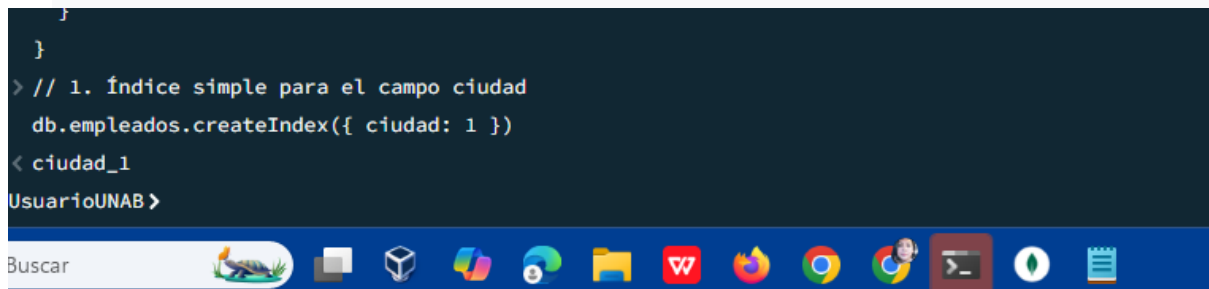
Lo que deba hacer cada grupos es:

- Índice simple para el campo **ciudad**.
- Índice compuesto para **ciudad** y **edad**.
- Índice único en el campo **correo**.
- Índice de texto en el campo **cargo**.

### Evidencias realizadas

- Índice simple para el campo **ciudad**.

```
}  
}  
> // 1. Índice simple para el campo ciudad  
db.empleados.createIndex({ ciudad: 1 })  
< ciudad_1  
UsuarioUNAB >
```



- Índice compuesto para **ciudad** y **edad**.

```
> // 2. Índice compuesto para ciudad y edad  
db.empleados.createIndex({ ciudad: 1, edad: 1 })  
< ciudad_1_edad_1  
UsuarioUNAB >
```

- Índice único en el campo correo.

```
> // 3. índice único en el campo correo
db.empleados.createIndex({ correo: 1 }, { unique: true })
< correo_1
UsuarioUNAB >
```

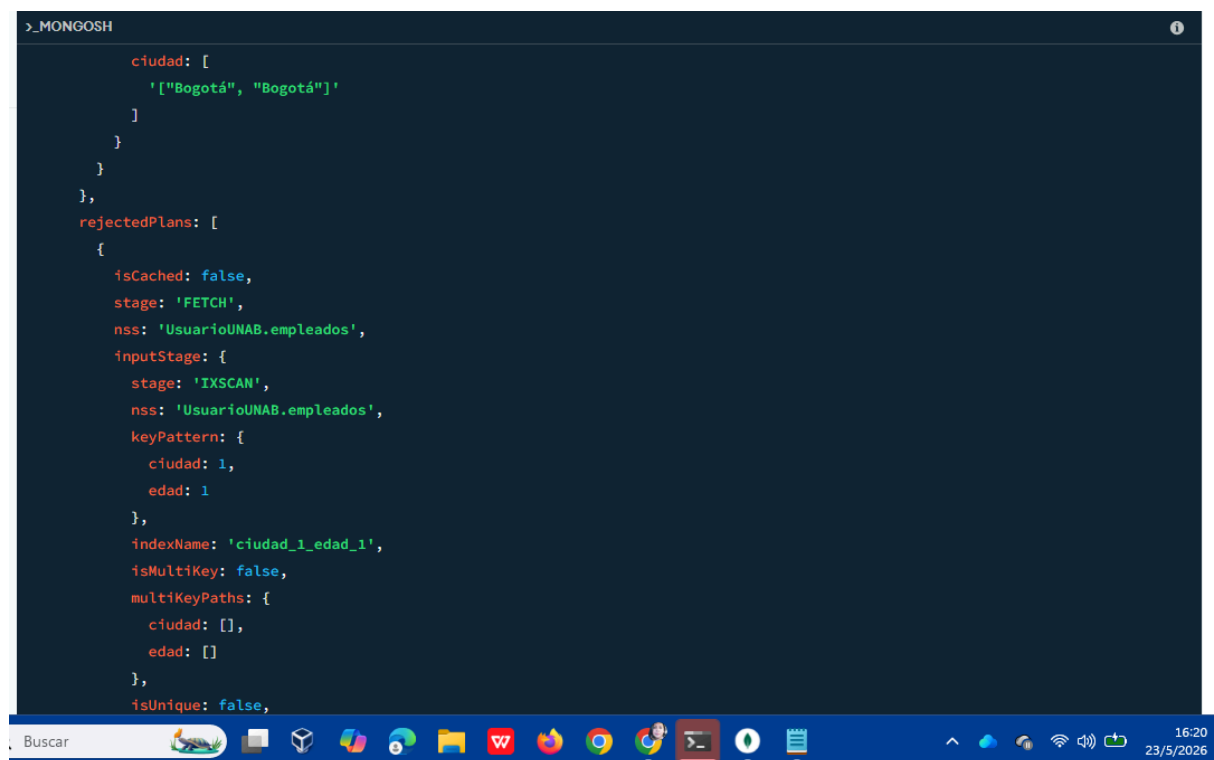
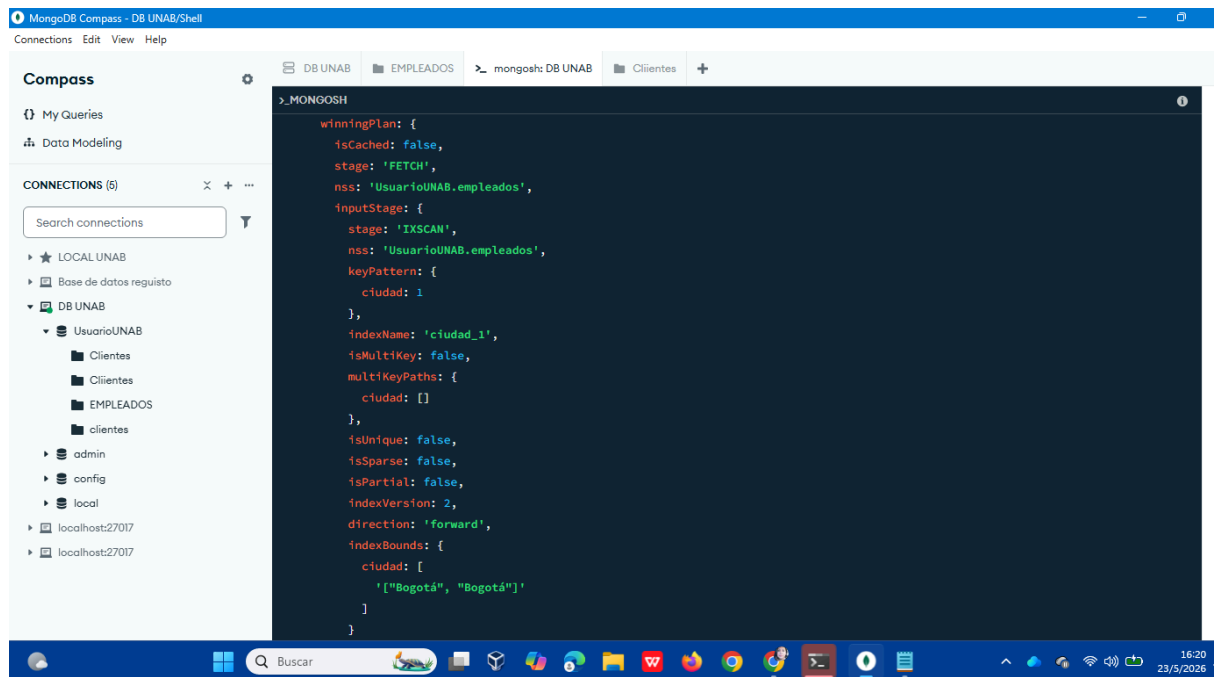
- Índice de texto en el campo cargo.

```
> // 4. índice de texto en el campo cargo
db.empleados.createIndex({ cargo: "text" })
< cargo_text
UsuarioUNAB >
```

## Comprobación de índices creados

```
db.empleados.find({ ciudad: "Bogotá" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'UsuarioUNAB.empleados',
    parsedQuery: {
      ciudad: {
        '$eq': 'Bogotá'
      }
    },
  },
  indexFilterSet: false,
  queryHash: 'EF9EB24E',
  planCacheShapeHash: 'EF9EB24E',
  planCacheKey: '31CA7BC2',
  optimizationTimeMillis: 6,
  maxIndexedOrSolutionsReached: false,
  maxIndexedAndSolutionsReached: false,
  maxScansToExplodeReached: false,
  prunedSimilarIndexes: false,
```





```

>_MONGOSH
    ciudad: [],
    edad: []
  },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: {
    ciudad: [
      '["Bogotá", "Bogotá"]'
    ],
    edad: [
      '[MinKey, MaxKey]'
    ]
  }
}
],
},
executionStats: {
  executionSuccess: true,
  nReturned: 0,
  executionTimeMillis: 11,
  totalKeysExamined: 0.
}

```

```

>_MONGOSH
totalDocsExamined: 0,
executionStages: {
  isCached: false,
  stage: 'FETCH',
  nReturned: 0,
  executionTimeMillisEstimate: 0,
  works: 2,
  advanced: 0,
  needTime: 0,
  needYield: 0,
  saveState: 1,
  restoreState: 1,
  isEOF: 1,
  nss: 'UsuarioUNAB.empleados',
  docsExamined: 0,
  alreadyHasObj: 0,
  inputStage: {
    stage: 'IXSCAN',
    nReturned: 0,
    executionTimeMillisEstimate: 0,
    works: 1,
    advanced: 0,
    needTime: 0,
    needYield: 0,
    saveState: 1.
  }
}

```

```
DB UNAB EMPLEADOS mongosh: DB UNAB Clientes +
>_MONGOSH

  restoreState: 1,
  isEOF: 1,
  nss: 'UsuarioUNAB.empleados',
  keyPattern: {
    ciudad: 1
  },
  indexName: 'ciudad_1',
  isMultiKey: false,
  multiKeyPaths: {
    ciudad: []
  },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: {
    ciudad: [
      '["Bogotá", "Bogotá"]'
    ]
  },
  keysExamined: 0,
  seeks: 1,
  dupsTested: 0,
  dupsDropped: 0.
```

```
DB UNAB EMPLEADOS mongosh: DB UNAB Clientes +
>_MONGOSH

  peakTrackedMemBytes: 0
}
},
queryShapeHash: '113CF96C5B7CDEB0FAAE3B687E96E3B948B816654A824CF0B94815D10F4A9D3E',
command: {
  find: 'empleados',
  filter: {
    ciudad: 'Bogotá'
  },
  '$db': 'UsuarioUNAB'
},
serverInfo: {
  host: 'Arelytino',
  port: 27017,
  version: '8.3.1',
  gitVersion: '094e631246d5b5bee5bd5f20b12f882bc8e286ea'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600.
```

```
DB UNAB | EMPLEADOS | mongosh: DB UNAB | Clientes | +
>_MONGOSH
  ciudad: 'Bogotá'
},
  '$db': 'UsuarioUNAB'
},
serverInfo: {
  host: 'Arelytino',
  port: 27017,
  version: '8.3.1',
  gitVersion: '094e631246d5b5bee5bd5f20b12f882bc8e286ea'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
UsuarioUNAB>
```

## Investigación técnica:

|                     |                             |
|---------------------|-----------------------------|
| ¿Qué es un índice?  | Explicación técnica         |
| Ventajas            | Beneficios en rendimiento   |
| Desventajas         | Impacto en almacenamiento y |
| Índices en Mongo DB | escrituras                  |
| Casos reales        | Tipos existentes            |
|                     | Uso empresarial             |

¿Qué es un índice?

Un índice en MongoDB es una estructura de datos especial que almacena una pequeña porción de los datos de una colección en un formato fácil de recorrer. Funciona igual que el índice de un libro: en lugar de leer página por página, vas directo a la página que necesitas. Sin índice, MongoDB hace un COLLSCAN o escaneo completo de todos los documentos.

### **Ventajas de usar índices en MongoDB:**

#### **1. Consultas mucho más rápidas**

Sin índice MongoDB revisa documento por documento. Con índice va directo al dato. En colecciones grandes pasas de 2-5 segundos a 2-10 ms.

#### **2. Menos consumo de CPU y RAM**

Al revisar menos documentos, el servidor trabaja menos. Eso baja el costo en servidores en producción.

#### **3. Ordenamiento eficiente**

sort() sin índice obliga a MongoDB a cargar todo en memoria y ordenar. Con índice ya viene ordenado, así que no hay límite de 32MB en sort.

#### **4. Búsquedas de texto nativas**

El índice tipo text te permite hacer \$text: { \$search: "palabra" } sin usar regex lentos.

#### **5. Garantiza unicidad de datos**

Con { unique: true } evitas duplicados en campos como correo, cedula, username automáticamente.

## 6. Mejora en JOINS y agregaciones

\$lookup y \$group son mucho más rápidos si los campos de unión tienen índice.

**Conclusión:** Las ventajas aplican solo a lecturas y consultas. Para escrituras masivas, cada índice extra ralentiza un poco los insert y update porque también hay que actualizarlo.

## Desventajas de los índices en MongoDB:

### 1. Ocupan espacio en disco

Cada índice es una copia extra de los datos indexados. Un índice puede pesar entre 10% y 80% del tamaño de la colección. Si creas 5 índices, tu BD puede duplicar su tamaño.

### 2. Hacen más lentas las escrituras

Cada insert, update y delete debe actualizar también todos los índices de la colección. Si tienes 4 índices, MongoDB hace 4 escrituras extra por cada documento. En cargas masivas se nota.

### 3. Consumen RAM

MongoDB intenta mantener los índices más usados en RAM para que sean rápidos. Si creas índices que no usas, estás desperdiciando memoria que podría usarse para cachear datos reales.

4. Pueden empeorar el rendimiento si están mal diseñados

Un índice compuesto mal ordenado no se usa. Peor aún, MongoDB puede elegir un índice ineficiente y hacer la consulta más lenta que sin índice. Por eso se usa `explain()` para verificar.

5. Mantenimiento y complejidad

Demasiados índices hacen difícil saber cuál se está usando. Borrar o modificarlos requiere tiempo y puede bloquear la colección en producción.

## **En MongoDB existen 7 tipos principales de índices**

1. Índice Simple o Single Field

Indexa un solo campo. Es el más común.

```
db.usuarios.createIndex({ email: 1 }) // 1 = ascendente, -1 = descendente
```

Sirve para `find({ email: "x" })` y `sort({ email: 1 })`.

2. Índice Compuesto o Compound

Indexa varios campos en un orden específico. El orden importa mucho.

```
db.ventas.createIndex({ cliente_id: 1, fecha: -1 })
```

Acelera consultas como `find({ cliente_id: 123, fecha: { $gt: ISODate() } })`. Si inviertes el orden, no sirve.

3. Índice Único o Unique

Evita valores duplicados en ese campo.

```
db.usuarios.createIndex({ cedula: 1 }, { unique: true })
```

MongoDB te tira error si intentas insertar 2 documentos con la misma cédula

#### 4. Índice de Texto o Text

Permite búsquedas de texto completo con stemming y stop words.

```
db.productos.createIndex({ descripcion: "text" })
```

Usas `find({ $text: { $search: "laptop gamer" } })`. Solo puede haber 1 índice de texto por colección.

#### 5. Índice TTL o Time To Live

Borra documentos automáticamente después de cierto tiempo. Útil para sesiones, logs, OTP.

```
db.sesiones.createIndex({ creadoEn: 1 }, { expireAfterSeconds: 3600 })
```

Los docs se eliminan 1 hora después de `creadoEn`.

#### 6. Índice Geospatial

Para consultas de ubicación: "dame los restaurantes a 2km de mí".

Hay 2 tipos:

- 2dsphere: Para coordenadas de la Tierra. El más usado.

- 2d: Para planos 2D planos, casi en desuso.

```
db.locales.createIndex({ ubicacion: "2dsphere" })
```

#### 7. Índice Multikey

No se crea manualmente. MongoDB lo crea solo cuando indexas un campo que es un array.

```
db.posts.createIndex({ tags: 1 })
```



Ahora puede buscar rápido `find({ tags: "mongodb" })` aunque tags sea `["mongodb", "db", "nosql"]`.

### **Explicación:**

También existe el índice `_id_`, que MongoDB crea automáticamente en toda colección y no se puede borrar.

Explicación técnica: cómo los índices dan beneficios de rendimiento en MongoDB

Sin índice, MongoDB hace un COLLSCAN. Eso significa recorrer toda la colección documento por documento en disco/RAM hasta encontrar los que cumplen la condición. Complejidad  $O(n)$ .

**Con índice, MongoDB hace un IXSCAN. Usa un árbol B-Tree para saltar directo a los datos que cumplen. Complejidad  $O(\log n)$ .**

#### 1. Por qué el árbol B-Tree es rápido

El índice guarda los valores ordenados y un puntero al documento en disco. El árbol B-Tree está balanceado, entonces para buscar un valor hace 15-20 comparaciones aunque haya 10 millones de documentos.

Ejemplo: Buscar edad: 25 sin índice = revisar 1M docs. Con índice = 20 saltos en el árbol + 1 acceso al doc.

#### 2. Menos I/O de disco

Solo se leen las páginas del índice y los documentos que coinciden.  
`explain("executionStats")` lo muestra como `totalDocsExamined` vs `totalDocsScanned`.

- Sin índice: scanned = examined = 50000
- Con índice: scanned = 50, examined = 5

Menos I/O = menos tiempo de espera.

### 3. Ordenamiento sin cargar en memoria

Cuando haces `sort({ fecha: -1 })` sin índice, MongoDB carga todos los docs en RAM, ordena y devuelve. Límite de 32MB. Si se pasa, truena.

Con índice en fecha, los datos ya salen ordenados del árbol. No hay carga en memoria ni límite. Por eso en producción todo sort lleva índice.

### 4. Cubriendo consultas - Covered Queries

Si la consulta pide solo campos que están en el índice, MongoDB ni siquiera toca los documentos. Responde directo desde el índice. Es lo más rápido posible.

Ejemplo: `db.usuarios.find({ ciudad: "Bogotá" }, { _id: 0, ciudad: 1 })` con índice `{ciudad: 1}` es covered.

### 5. Impacto en agregaciones y JOINS

`$lookup`, `$match`, `$group` usan índices si los campos de unión están indexados. Un `$lookup` sin índice hace un escaneo anidado  $O(n*m)$ . Con índice baja a  $O(n \log m)$ .

Dato real: En una colección de 2M documentos, buscar por campo no indexado tarda 1.2s. Con índice compuesto correcto tarda 4ms. Eso es 300x más rápido.

La clave está en `explain("executionStats")`: si ves `stage: "IXSCAN"` y `totalDocsExamined` bajo, el índice está trabajando. Si ves `COLLSCAN`, no se está usando.

## **1. Impacto en almacenamiento**

Cada índice es una estructura B-Tree separada que MongoDB guarda en disco y en RAM.

Qué guarda:

- El valor del campo indexado, ordenado
- Un puntero de 8 bytes al documento original
- Overhead del árbol B-Tree

Cuánto pesa:

Para una colección de 100k documentos, un índice en un campo string de 15 caracteres pesa aprox 8-12 MB.

Si creas 4 índices, es común que `totalIndexSize` sea 40-80% del `dataSize` de la colección.

Puedes verificarlo con:

```
db.empleados.stats().indexSizes
```

Ahí ves el peso de cada índice por separado.

RAM: MongoDB carga los índices activos en memoria para que las búsquedas sean rápidas. Si tienes 10 índices pero solo usas 2, estás desperdiciando RAM que podría usarse para cachear datos.

## 2. Impacto en escrituras

Cada insert, update y delete debe actualizar todos los índices de la colección.

Insert:

Sin índices: 1 escritura en la colección.

Con 4 índices: 1 escritura en la colección + 4 escrituras en los árboles B-Tree.

Update:

Si modificas un campo indexado, MongoDB hace 2 operaciones por índice: borra la entrada antigua y crea la nueva.

Delete:

Debe recorrer y borrar la entrada de cada índice.

Resultado práctico:

- 1 índice  $\approx$  8-15% más lento el insert
- 4 índices  $\approx$  30-60% más lento el insert
- 8 índices  $\approx$  escrituras se vuelven 2-3x más lentas

### 3. Cuándo vale la pena

Los índices solo valen la pena si la mejora en lecturas compensa la pérdida en escrituras.

Vale la pena cuando:

- Lees 100 veces más de lo que escribes
- La consulta pasa de 1s a 5ms y el usuario lo nota
- El campo se usa en where, sort, lookup, group

No vale la pena cuando:

- El campo nunca se usa en consultas
- La colección es solo de escritura tipo logs
- Tienes 20 índices "por si acaso"

### Conclusión

En la práctica de EMPLEADOS, los 4 índices tienen sentido porque ciudad, correo y cargo son campos típicos de búsqueda. Si agregaras un índice en salario y nunca lo filtras, estarías pagando el costo de escritura sin ganar nada.

### Casos reales de uso de índices en empresas con MongoDB

#### Ejemplos:

#### 1. E-commerce: búsqueda de productos

Caso: Mercado Libre, Rappi, Linio

Índice: { categoría: 1, precio: 1, calificación: -1 } compuesto + { descripción: "text" }

Por qué: Cuando filtras "Laptops > \$800, ordenado por rating", sin índice hace COLLSCAN sobre 2M productos. Con índice compuesto responde en 20ms. El

índice text permite buscar "auriculares bluetooth cancelación ruido" sin usar regex lento.

## **2. Banca y Fintech: prevención de fraude**

Caso: Nubank, Davivienda con Neo

Índice: { cuenta\_id: 1, fecha: -1 } único en transacciones

Por qué: Necesitan consultar "todas las transacciones de esta cuenta en los últimos 7 días" para detectar patrones sospechosos. El índice compuesto permite recorrer solo los últimos docs de esa cuenta. El unique evita duplicados en transacciones críticas.

## **3. Apps de delivery: geolocalización**

Caso: Uber Eats, Didi Food, Rappi

Índice: { ubicación: "2dsphere" } en restaurantes y repartidores

Por qué: "Dame restaurantes a 2km de mi ubicación" se resuelve con un índice geoespacial. Sin índice tendría que calcular distancia a todos los restaurantes del país. Con índice solo revisa los que están en el área del árbol geo.

## **4. SaaS y logs: TTL para limpieza automática**

Caso: Datadog, MongoDB Atlas mismo

Índice: { creado En: 1 } con expireAfterSeconds: 2592000

Por qué: Guardan logs de eventos por 30 días. El índice TTL borra automáticamente docs viejos sin que tengas que correr un cron. Ahorra storage y mantiene la BD rápida.

## **5. RRHH y ERP: validación de datos**

Caso: SAP, Oracle HCM, cualquier sistema de nómina

Índice: { cedula: 1 } unique, { correo: 1 } unique

Por qué: Igual que tu índice en correo de EMPLEADOS. Evita que se registren 2 empleados con la misma cédula o correo por error humano. La validación pasa a nivel BD, no depende del código.

## **6. Redes sociales: feed y notificaciones**

Caso: LinkedIn, Twitter

Índice: { usuario\_id: 1, creado En: -1 } compuesto

Por qué: "Dame los últimos 20 posts de los usuarios que sigo". El índice permite hacer \$lookup + \$sort sin cargar millones de posts en memoria.

Patrón común:

Todas usan índices compuestos cuando filtran ordenan, e índices únicos para integridad de datos. Nadie indexa todo. Un servicio de 50M docs suele tener 8-12 índices bien pensados, no 200.